Лабораторная работа №1

Вычисление числа π с помощью ряда Лейбница

Работу выполнил: Щукин Георгий 11-401 • **Дата:** 08.10.2026 •

1. Краткое описание работы

Реализованы последовательная и параллельная версии программы вычисления числа π по ряду Лейбница $\pi = 4 \cdot \Sigma\ (-1)^i/(2i+1)$. Параллельная версия использует OpenMP и директиву `#pragma omp parallel for reduction(+:sum)`. Проведены две серии экспериментов: исследование влияния количества потоков при фиксированном $n = 10^9$ и исследование влияния размера задачи при фиксированных 8 потоках.

2. Результаты экспериментов

Эксперимент 1. Влияние количества потоков, $n = 10^9$.

Число потоков	Время послед., с	Время парал., с	Ускорение
1	0.920703	0.876969	1.05
2	0.883403	0.465911	1.90
4	0.895078	0.245093	3.65
8	0.916198	0.142753	6.42

Таблица 1. Зависимость времени и ускорения от числа потоков.

Эксперимент 2. Влияние размера задачи, 8 потоков.

n	Время послед., с	Время парал., с	Ускорение
10^7	0.009009	0.001781	5.06
10^8	0.089250	0.015860	5.63
10^9	0.907503	0.145524	6.24
10^{10}	8.939841	1.516561	5.89

Таблица 2. Зависимость времени и ускорения от размера задачи.

3. Вывод

При увеличении числа потоков от 1 до 8 время выполнения монотонно уменьшается, а ускорение растёт практически линейно и достигает максимума ≈ 6.4 при 8 потоках, что соответствует числу физических ядер процессора. На каждом шаге удвоения числа потоков время падало примерно в 1.7–2 раза, то есть распараллеливание шло почти идеально. Заметного замедления роста ускорения в диапазоне 1–8 потоков не наблюдается, так как точка насыщения совпадает с числом физических ядер (8). Превышение этого значения (например, 16 потоков) привело бы к падению ускорения из-за накладных расходов OpenMP на создание потоков и операцию.

Посмотрим как влияет размер задачи на эффективность OpenMP. На малом $n = 10^7$ ускорение составило лишь 5.06 из-за того, что доля накладных расходов на распараллеливание относительно велика. С ростом n до 10^9 ускорение повышается до 6.24, поскольку время полезных вычислений растёт быстрее накладных расходов. На $n = 10^{10}$ ускорение слегка снижается (5.89). Все потому, что сказывается рост нагрузки на подсистему памяти и накопление погрешности суммирования при большом числе слагаемых.